

# ABC455 G - Balanced Subarrays 题解

## 题意概括

定义一个连续子数组是“平衡的”，当且仅当其中所有出现过的整数，其出现次数都完全相同。

对每个给定的  $B_k$ ，需要统计两类子数组数量：

1. 每个出现过的整数都恰好出现  $B_k$  次；
2. 恰好有  $B_k$  个不同整数出现。

## 解题思路

这题需要分别统计上面两类答案。  $K \leq 10$ ，所以可以对每个  $B_k$  单独做一遍线性或线性对数时间的扫描。

实现里使用了双 64 位随机哈希，把“很多个值的计数条件”压缩成前缀哈希相等判定。它是竞赛里很常见的 Monte Carlo 做法，碰撞概率可以忽略。

### 一、统计“每个数恰好出现 B 次”

固定一个  $B$ 。

如果某个子数组满足题意，那么其中每个值的出现次数只能是：

- 0 次；
- $B$  次。

先用双指针扫右端点  $r$ ，维护最小左端点  $L_r$ ，使得区间  $[L_r, r]$  中没有任何值出现超过  $B$  次。

于是对于所有  $l \geq L_r$  的子数组  $[l, r]$ ：

- 每个值的出现次数都在  $0 \dots B$  之间；
- 如果再额外满足“每个值的出现次数都对  $B$  取模为 0”，那就只能是 0 或  $B$ ，也就恰好满足第一类要求。

所以问题变成：如何快速判断子数组里每个值的出现次数是否都被  $B$  整除？

做法是给每个值设计一个长度为  $B$  的哈希循环：

- 第 0 次出现给一个哈希值；
- 第 1 次出现给一个哈希值；
- ...

- 第  $B - 1$  次出现给一个哈希值;
- 并且让这一整轮哈希和为  $\theta$ 。

这样同一个值每出现  $B$  次, 贡献总和就归零。于是:

- 若某个子数组中某个值出现次数是  $B$  的倍数, 它对总哈希贡献为  $\theta$ ;
- 反之, 一般不为  $\theta$ 。

于是只要统计前缀哈希相等的前缀对, 就能找出满足 “所有计数都对  $B$  取模为  $\theta$ ” 的子数组, 再配合左端点下界  $L_r$  过滤, 就得到第一类答案。

## 二、统计 “恰好有 $B$ 个不同整数出现”

还是固定一个  $B$ 。

对每个右端点  $r$ , 用两套双指针维护:

- $lb$ : 使  $[lb, r]$  中不同整数个数至多为  $B$ ;
- $lb1$ : 使  $[lb1, r]$  中不同整数个数至多为  $B - 1$ 。

于是恰好有  $B$  个不同整数的左端点  $l$  恰好构成一个连续区间:

$$lb \leq l \leq lb1 - 1$$

更重要的是, 这个区间里的所有子数组都包含同一组不同整数。记这一组整数的哈希和为  $H$ 。

再设:

- $S_i$ : 前  $i$  个数的 “值哈希” 前缀和;
- 当前子数组长度为  $len = r - l + 1$ ;
- 既然它是平衡的, 且恰有  $B$  个不同整数, 那么每个值都出现  $len / B$  次。

因此:

$$S_r - S_{l-1} = H \times \frac{len}{B}$$

移项后可得:

$$B \times S_r - H \times r = B \times S_{l-1} - H \times (l - 1)$$

所以对固定的  $r$  和固定的  $H$ , 只要统计区间  $[lb - 1, lb1 - 2]$  中有多少个前缀下标  $p = l - 1$  满足:

$$B \times S_p - H \times p = B \times S_r - H \times r$$

就能得到第二类答案。

随着  $r$  从左到右扫描:

- 如果这组  $B$  个不同整数没有变化，就继续在频率表里增删前缀即可；
- 如果这组整数变了，就重建当前频率表。

这样总复杂度仍然是线性的。

## 正确性说明

### 第一类答案

双指针维护的  $L_r$  保证：对任意  $l \geq L_r$ ，子数组  $[l, r]$  中每个值的出现次数都不超过  $B$ 。

此时：

- 若某个值在  $[l, r]$  中出现次数是  $B$  的倍数，那么它只能是  $0$  或  $B$ ；
- 于是“所有值的出现次数都被  $B$  整除”等价于“每个出现过的值都恰好出现  $B$  次”。

哈希循环整轮和为  $0$ ，所以前缀哈希相等恰好对应“每个值的出现次数都对  $B$  取模为  $0$ ”（碰撞概率极低，可忽略）。因此统计出的就是第一类答案。

### 第二类答案

对固定右端点  $r$ ，左端点区间  $l_b \leq l \leq l_{b+1} - 1$  恰好对应“子数组中不同整数个数正好为  $B$ ”。

这一整段左端点对应的不同整数集合相同，记为  $U$ ，它的哈希和记为  $H$ 。

若某个子数组平衡，那么设每个值都出现  $t$  次，则：

- 子数组长度  $len = B * t$
- 子数组值哈希和  $S_r - S_{l-1} = t * H$

这恰好等价于：

$$B \times S_r - H \times r = B \times S_{l-1} - H \times (l-1)$$

反过来，只要这个等式成立，且子数组里恰有  $B$  个不同值，那么在随机哈希不碰撞的前提下，就对应每个值出现次数相同。

因此第二部分的统计也正确。

## 复杂度

设总长度为  $N$ 。

对每个  $B_k$  做两次线性扫描。

- 时间复杂度： $O(KN)$  期望时间

- 空间复杂度：O(N)

在  $K \leq 10$ 、所有测试总  $N \leq 2 * 10^5$  的条件下可以通过。

## 参考实现

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using ull = unsigned long long;

struct HashValue{
    ull x, y;

    bool operator == (const HashValue &other) const{
        return x == other.x && y == other.y;
    }
};

HashValue operator + (const HashValue &a, const HashValue &b){
    return {a.x + b.x, a.y + b.y};
}

HashValue operator - (const HashValue &a, const HashValue &b){
    return {a.x - b.x, a.y - b.y};
}

HashValue operator * (const HashValue &a, ull k){
    return {a.x * k, a.y * k};
}

struct HashHasher{
    size_t operator () (const HashValue &h) const noexcept{
        ull z = h.x ^ (h.y + 0x9e3779b97f4a7c15ULL + (h.x << 6) + (h.x >> 2));
        return (size_t)z;
    }
};

ull splitmix64(ull x){
    x += 0x9e3779b97f4a7c15ULL;
    x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9ULL;
```

```

    x = (x ^ (x >> 27)) * 0x94d049bb133111ebULL;
    return x ^ (x >> 31);
}

HashValue make_hash(ull seed){
    return {splitmix64(seed), splitmix64(seed ^ 0x9e3779b97f4a7c15ULL)};
}

// 第二部分会频繁用到这个表达式:
// B * prefix[idx] - set_hash * idx
HashValue get_key(const vector<HashValue> &prefix, int idx, int b, const
    HashValue &set_hash){
    return prefix[idx] * (ull)b - set_hash * (ull)idx;
}

// 第一类答案: 每个出现过的值都恰好出现 b 次
ll count_each_exactly_b(const vector<int> &a, int n, int b){
    // 给每个值构造一个长度为 b 的哈希循环, 整轮和为 0
    vector<array<HashValue, 10>> cycle(n + 1);
    for(int x=1; x<=n; x++){
        HashValue sum = {0, 0};
        for(int r=0; r<b - 1; r++){
            cycle[x][r] = make_hash(((ull)b << 32) ^ ((ull)x << 8) ^ (ull)r ^
                0xabcdefULL);
            sum = sum + cycle[x][r];
        }
        cycle[x][b - 1] = {0ULL - sum.x, 0ULL - sum.y};
    }

    // prefix[i] 记录前 i 个数在这套“模 b 循环哈希”下的前缀和
    vector<int> occ(n + 1, 0);
    vector<HashValue> prefix(n + 1, {0, 0});
    for(int i=1; i<=n; i++){
        int x = a[i];
        int idx = occ[x] % b;
        prefix[i] = prefix[i - 1] + cycle[x][idx];
        occ[x]++;
    }

    vector<int> cnt(n + 1, 0);
    unordered_map<HashValue, ll, HashHasher> freq;

```

```

freq.reserve(2 * n + 5);

int left = 1;
int left_prefix = 0;
freq[prefix[0]] = 1;

ll ans = 0;
for(int r=1; r<=n; r++){
    // 双指针保证当前窗口内每个值出现次数都不超过 b
    cnt[a[r]]++;
    while(cnt[a[r]] > b){
        cnt[a[left]]--;
        left++;
    }

    // 只保留合法左端点之前缀
    int need_left = left - 1;
    while(left_prefix < need_left){
        auto it = freq.find(prefix[left_prefix]);
        it->second--;
        if(it->second == 0){
            freq.erase(it);
        }
        left_prefix++;
    }

    auto it = freq.find(prefix[r]);
    if(it != freq.end()){
        ans += it->second;
    }
    freq[prefix[r]]++;
}

return ans;
}

// 第二类答案：恰好有 b 个不同整数出现，且这些值出现次数都相同
ll count_distinct_b(const vector<int> &a, int n, int b, const vector<HashValue>
    &value_hash, const vector<HashValue> &prefix){

    vector<int> cnt_b(n + 1, 0), cnt_b1(n + 1, 0);

```

```

int distinct_b = 0, distinct_b1 = 0;
int lb = 1, lb1 = 1;

HashValue window_hash = {0, 0};

unordered_map<HashValue, ll, HashHasher> freq;
freq.reserve(2 * n + 5);

bool active = false;
HashValue current_set_hash = {0, 0};
int cur_l = 0, cur_r = -1;

ll ans = 0;

for(int r=1; r<=n; r++){
    // lb 维护“不同值个数至多为 b”的最小左端点
    if(cnt_b[a[r]] == 0){
        distinct_b++;
    }
    cnt_b[a[r]]++;
    while(distinct_b > b){
        cnt_b[a[lb]]--;
        if(cnt_b[a[lb]] == 0){
            distinct_b--;
        }
        lb++;
    }

    // lb1 维护“不同值个数至多为 b - 1”的最小左端点
    if(cnt_b1[a[r]] == 0){
        distinct_b1++;
        window_hash = window_hash + value_hash[a[r]];
    }
    cnt_b1[a[r]]++;
    while(distinct_b1 > b - 1){
        int x = a[lb1];
        cnt_b1[x]--;
        if(cnt_b1[x] == 0){
            distinct_b1--;
            window_hash = window_hash - value_hash[x];
        }
    }
}

```

```

        lb1++;
    }

    // 满足“恰好 b 个不同值”的左端点范围是 [lb, lb1 - 1]
    int need_l = lb - 1;
    int need_r = lb1 - 2;
    if(need_l > need_r){
        active = false;
        freq.clear();
        continue;
    }

    // 这一整段左端点对应的不同整数集合相同，它们的值哈希和记为 set_hash
    HashValue set_hash = window_hash + value_hash[a[lb1 - 1]];

    if(!active || !(set_hash == current_set_hash)){
        active = true;
        current_set_hash = set_hash;
        freq.clear();

        // 集合变了，重建这一段前缀键值的频率表
        cur_l = need_l;
        cur_r = need_r;
        for(int i=cur_l; i<=cur_r; i++){
            freq[get_key(prefix, i, b, current_set_hash)]++;
        }
    } else {
        // 集合没变，只需把有效区间滑动到新的 [need_l, need_r]
        while(cur_l < need_l){
            HashValue key = get_key(prefix, cur_l, b, current_set_hash);
            auto it = freq.find(key);
            it->second--;
            if(it->second == 0){
                freq.erase(it);
            }
            cur_l++;
        }
        while(cur_r < need_r){
            cur_r++;
            freq[get_key(prefix, cur_r, b, current_set_hash)]++;
        }
    }
}

```

```

    }

    // 匹配  $B * S_p - H * p = B * S_r - H * r$  的前缀个数
    HashValue target = get_key(prefix, r, b, current_set_hash);
    auto it = freq.find(target);
    if(it != freq.end()){
        ans += it->second;
    }
}

return ans;
}

int main(){

    int T;
    cin >> T;

    while(T--){
        int n, k;
        cin >> n >> k;

        // a 是原数组, b[i] 是每次要查询的 B_k
        vector<int> a(n + 1), b(k);
        for(int i=1; i<=n; i++){
            cin >> a[i];
        }
        for(int i=0; i<k; i++){
            cin >> b[i];
        }

        // 为每个值准备独立随机哈希, 再求普通值哈希前缀和
        vector<HashValue> value_hash(n + 1, {0, 0});
        for(int x=1; x<=n; x++){
            value_hash[x] = make_hash((ull)x * 1234567ULL + 890123ULL);
        }

        vector<HashValue> prefix(n + 1, {0, 0});
        for(int i=1; i<=n; i++){
            prefix[i] = prefix[i - 1] + value_hash[a[i]];
        }
    }
}

```

```
// 每个  $B_k$  独立统计两类答案
for(int i=0; i<k; i++){
    ll ans1 = count_each_exactly_b(a, n, b[i]);
    ll ans2 = count_distinct_b(a, n, b[i], value_hash, prefix);
    cout << ans1 << ' ' << ans2 << '\n';
}

return 0;
}
```